

Trabalho Prático – Entrega Final 08/06/2026

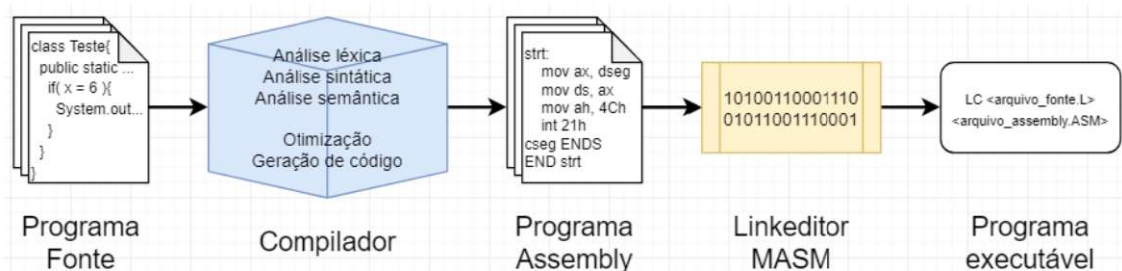
Desenvolvimento de um compilador em linguagem imperativa simplificada

I – Objetivo

O objetivo do trabalho prático é desenvolver um compilador completo que seja capaz de traduzir programas escritos na linguagem fonte “BRL” para um subconjunto de instruções da linguagem Assembly (80x86). Tanto a linguagem fonte quanto a geração do assembler deverão ser realizadas durante o semestre. Para o final do TP, o compilador desenvolvido deve produzir um arquivo texto com extensão ASM, o qual será convertido em linguagem de máquina pelo montador MASM e executado via linha de comando. Em caso de detecção de erros no programa fonte, o compilador deve reportar o erro encontrado e terminar o processo de compilação.

As mensagens de erros deverão seguir as especificações do formato rigorosamente.

O programa executável do compilador desenvolvido deve ter o nome “BRL”, e também deve ter dois parâmetros, recebendo os argumentos por linha de comando: 1) nome completo do programa fonte a ser compilado com extensão .LC; e 2) nome completo do programa assembly com extensão .ASM a ser gerado.



II – Definição da linguagem fonte do compilador

O compilador desenvolvido utiliza uma linguagem imperativa simplificada, que possui características das linguagens C e Pascal. A linguagem trata 4 tipos básicos de variáveis:

<i>inteiro</i>	O tipo <i>inteiro</i> é um escalar que varia de -2^{31} a $2^{31}-1$, ocupando 4 bytes (32 bits).
<i>caractere</i>	O tipo <i>caractere</i> é um arranjo que pode conter até 255 caracteres úteis e quando armazenado em memória, é finalizado pelo caractere '\$', ocupando 512 bytes de memória.
<i>logico</i>	O tipo <i>logico</i> pode ter os valores TRUE e FALSE, ocupando um byte de memória (0h para falso e FFh para verdadeiro).
<i>real</i>	O tipo <i>real</i> é um escalar de ponto flutuante com 7 dígitos de precisão, que varia de 1.5×10^{-45} a 3.4×10^{38} , ocupando 4 bytes (32) bits.

No arquivo fonte, os caracteres/símbolos permitidos são: letra, dígito, espaço, sublinhado, ponto, vírgula, ponto-e-vírgula, dois-pontos, parênteses, colchetes, chaves, sinal de mais, sinal de menos, aspas, apóstrofo, barra, barra invertida, barra em pé (pipe), exclamação, interrogação, maior, menor e igual, além da quebra de linha (bytes 0Dh e 0Ah).

ATENÇÃO: Qualquer outro caractere é considerado inválido.

Variáveis do tipo *caractere* são delimitadas por aspas e não podem conter quebra de linha.

Os identificadores de constantes e variáveis são compostos de letras, dígitos e o sublinhado, começando necessariamente por uma letra ou sublinhado e podem ter no máximo 512 caracteres. A linguagem deve fazer distinção entre maiúsculas e minúsculas (*sensitive-case*).

As palavras a seguir são reservadas:

inteiro	caractere	logico	real	se	enquanto	leitura
escrita	ou	&&	==	:=	(	)
<	>	<>	>=	<=	,	+
-	*	/	;	:	inicio	fim
div	mod	verdadeiro	falso	senao	faca	

As instruções permitem atribuição para variáveis, entrada de valores pelo teclado e saída de valores para a tela, blocos (*inicio - fim*), estruturas de repetição (*enquanto*), estruturas de teste (*se - senao*), expressões aritméticas com inteiros e real, expressões lógicas e relacionais, além de atribuição, concatenação e comparação de igualdade entre caracteres. A ordem de precedência para as expressões deve ser:

- parênteses;
- negação lógica (*not*);
- multiplicação aritmética (*), lógica (&&) e divisão (/ , div);
- subtração (-), adição aritmética (+), lógica (ou) e concatenação de strings (+);
- comparação aritmética (==, <>, <,>, <=, >=) e entre caracteres (==).

Os comentários devem estar entre /* */ , tanto para o comentário de uma linha quanto para comentários de mais de uma linha.

A quebra de linha e o espaço devem ser usados como delimitadores de lexemas.

A estrutura básica do programa fonte deve seguir a seguinte estrutura:

Declarações e Bloco

Siga, atentamente, a descrição informal da sintaxe das declarações e comandos da linguagem a ser desenvolvida:

- O programa tem o formato: **inicio identificador ;**
 declaracoes
 instrucoes
 fim

Dentro do bloco *inicio* e *fim*, podem ocorrer uma ou diversas declarações e instruções, porém, caso existam instruções, então deve haver as declarações relacionadas às instruções. No entanto, pode não ocorrer nenhuma declaração e, conseqüentemente, nenhuma instrução.

2. Declaração de variáveis: ***id [, id] : tipo ;***
A representação ***[, id]*** significa que, se houver mais de um id, estes devem ser separados por vírgula. Se houve mais de uma declaração, estas devem ser separadas por ponto-e-vírgula.
3. O *tipo* das variáveis são: *inteiro, caractere, logico* ou *real*.
4. As instruções tem o formato: ***instrucao [; instrucao]***
A representação ***[; instrucao]*** significa que, se houver mais de uma instrução, estes devem ser separados por ponto-e-vírgula.
5. O conjunto de instruções podem ser:
 - a) A atribuição tem o formato: ***id := EXP;***
 - b) A instrução ***se*** tem o formato: ***se EXP entao inicio instrucoes fim [senao inicio instrucao fim]***
 - c) A instrução ***leia*** tem o formato: ***leia (id [, id])***
 - d) A instrução ***escreva*** tem o formato: ***escreva (EXP)***
 - e) A instrução enquanto tem o formato: ***enquanto EXP faca inicio instrucoes fim***
6. O bloco EXP tem o formato: EXP **RELOP** EXP | EXP **ADDOP** EXP | EXP **MULOP** EXP | ID | CONST | (EXP)
 - a. **RELOP** possui o formato: EXP (**=** | **<** | **<=** | **>** | **>=** | **<>**) EXP
 - b. **ADDOP** possui o formato: EXP (**+** | **-** | **OU**) EXP
 - c. **MULOP** possui o formato: EXP (***** | **/** | **div** | **mod** | **&&**) EXP

Convenções Léxicas

1. Os IDs são definidos por **letras** maiúsculas e/ou minúsculas.
2. Os **dígitos** são definidos por números entre 0~9.
3. Cada **ID** deve, obrigatoriamente, começar com letra, podendo, ou não, ter uma combinação de letra e dígito na sequência.
4. Cada **CONST** pode ser um caractere ou uma sequência de caracteres, um dígito ou uma sequência de dígitos. Caso CONST seja dígito, então pode ter, ou não, o sinal de mais ou o sinal de menos anterior ao primeiro dígito.

III – Considerações gerais para todas as práticas:

1. O trabalho deverá ser feito em grupo, previamente definido, sem qualquer participação de outros grupos e/ou ajuda de terceiros. Cada aluno deve participar ativamente em todas as etapas do trabalho prático. Após serem designados os componentes dos grupos, estes não poderão ser alterados durante o semestre. O(A)s aluno(a)s que não tiverem feito grupos até o começo das implementações, ficarão sem a nota do trabalho prático.
2. A codificação do trabalho deve ser feita na linguagem Java de programação, em uma plataforma de desenvolvimento homologada e instalada nos laboratórios de Informática. Não poderão ser

utilizados bibliotecas gráficas ou qualquer recurso que não esteja previamente definido na descrição do trabalho prático pelo professor.

3. Os trabalhos devem ser postados na forma de um arquivo compactado no formato ZIP ou RAR, e deve ter o nome ou RA dos integrantes do grupo separados por sublinhado, por exemplo: *nome1_nome2_nome3_nome4.zip*. Obviamente, os códigos fontes devem estar no arquivo compactado, e também, devem conter o nome de todos os componentes do grupo nas primeiras linhas do arquivo principal.
4. A cópia de TPs, códigos na sua totalidade ou parcialmente copiados ou “encomendados”, serão zerados para todos os integrantes do(s) grupo(s) envolvido(s). É de responsabilidade do grupo manter o sigilo sobre a sua codificação, evitando que outros grupos tenham acesso a ele.
5. Será solicitada ao Colegiado uma advertência formal no caso de cópia por má fé.
6. Para a apresentação do trabalho prático, poderá haver perguntas relativas ao trabalho a qualquer integrante do grupo. Assim, todos os integrantes devem comparecer e serem capazes de responder a quaisquer perguntas e/ou alterar o código de qualquer parte do trabalho a pedido do professor, a fim de avaliar individualmente.
7. **ATENÇÃO:** A especificação do trabalho deve ser rigorosamente obedecida, principalmente com relação à interface com o usuário, uma vez que parte da correção será automatizada. Observe atentamente o nome do programa executável, os seus argumentos de entrada e formatos das mensagens. Os trabalhos com interfaces distintas das especificações poderão não ser avaliados.
8. A avaliação será baseada nos seguintes critérios:
 - Correção e robustez dos programas.
 - Conformidade às especificações prévias.
 - Clareza de codificação (comentários, endentação, nomes dos identificadores).
 - Organização dos arquivos no projeto o compilador.
 - Parametrizações dos métodos.
 - Apresentação individual dos integrantes.
 - Documentação do compilador desenvolvido.

Bom trabalho!